mod5_final_project

December 11, 2025

Assignment: Notebook for Graded Assessment

1 Introduction

Using this Python notebook you will:

1. Understand three Chicago datasets
2. Load the three datasets into three tables in a SQLIte database
3. Execute SQL queries to answer assignment questions

1.1 Understand the datasets

To complete the assignment problems in this notebook you will be using three datasets that are available on the city of Chicago's Data Portal:

1. Socioeconomic Indicators in Chicago
2. Chicago Public Schools
3. Chicago Crime Data

1.1.1 1. Socioeconomic Indicators in Chicago

This dataset contains a selection of six socioeconomic indicators of public health significance and a "hardship index," for each Chicago community area, for the years 2008 – 2012.

A detailed description of this dataset and the original dataset can be obtained from the Chicago Data Portal at:

https://data.cityofchicago.org/Health-Human-Services/Census-Data-Selected-socioeconomic-indicators-in-C/kn9c-c2s2

1.1.2 2. Chicago Public Schools

This dataset shows all school level performance data used to create CPS School Report Cards for the 2011-2012 school year. This dataset is provided by the city of Chicago's Data Portal.

A detailed description of this dataset and the original dataset can be obtained from the Chicago Data Portal at:

https://data.cityofchicago.org/Education/Chicago-Public-Schools-Progress-Report-Cards-2011-/9xs2-f89t

1.1.3 3. Chicago Crime Data

This dataset reflects reported incidents of crime (with the exception of murders where data exists for each victim) that occurred in the City of Chicago from 2001 to present, minus the most recent seven days.

A detailed description of this dataset and the original dataset can be obtained from the Chicago Data Portal at:

https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-present/ijzp-q8t2

1.1.4 Download the datasets

This assignment requires you to have these three tables populated with a subset of the whole datasets.

In many cases the dataset to be analyzed is available as a .CSV (comma separated values) file, perhaps on the internet.

Use the links below to read the data files using the Pandas library.

- Chicago Census Data

https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DB0201EN-SkillsNetwork/labs/FinalModule_Coursera_V5/data/ChicagoCensusData.csv?utm_medium=Exinflu SkillsNetwork-Channel-SkillsNetworkCoursesIBMDeveloperSkillsNetworkDB0201ENSkillsNetwork20127838-2021-01-01

- Chicago Public Schools

https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DB0201EN-SkillsNetwork/labs/FinalModule_Coursera_V5/data/ChicagoPublicSchools.csv?utm_medium=Exin SkillsNetwork-Channel-SkillsNetworkCoursesIBMDeveloperSkillsNetworkDB0201ENSkillsNetwork20127838-2021-01-01

- Chicago Crime Data

https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DB0201EN-SkillsNetwork/labs/FinalModule_Coursera_V5/data/ChicagoCrimeData.csv?utm_medium=Exinflu SkillsNetwork-Channel-SkillsNetworkCoursesIBMDeveloperSkillsNetworkDB0201ENSkillsNetwork20127838-2021-01-01

NOTE: Ensure you use the datasets available on the links above instead of directly from the Chicago Data Portal. The versions linked here are subsets of the original datasets and have some of the column names modified to be more database friendly which will make it easier to complete this assignment.

Execute the below code cell to install the required libraries

[1]:
```
!pip install pandas
!pip install ipython-sql prettytable

import prettytable
```

```
prettytable.DEFAULT = 'DEFAULT'
```

```
Collecting pandas
  Downloading pandas-2.3.3-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (91 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading
numpy-2.3.5-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(62 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/conda/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.12/site-
packages (from pandas) (2024.2)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-
packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading
pandas-2.3.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.4
MB)
                         12.4/12.4 MB
132.6 MB/s eta 0:00:00
Downloading
numpy-2.3.5-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.6
MB)
                         16.6/16.6 MB
142.4 MB/s eta 0:00:00
Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, numpy, pandas
Successfully installed numpy-2.3.5 pandas-2.3.3 tzdata-2025.2
Collecting ipython-sql
  Downloading ipython_sql-0.5.0-py3-none-any.whl.metadata (17 kB)
Collecting prettytable
  Downloading prettytable-3.17.0-py3-none-any.whl.metadata (34 kB)
Requirement already satisfied: ipython in /opt/conda/lib/python3.12/site-
packages (from ipython-sql) (8.31.0)
Requirement already satisfied: sqlalchemy>=2.0 in
/opt/conda/lib/python3.12/site-packages (from ipython-sql) (2.0.37)
Collecting sqlparse (from ipython-sql)
  Downloading sqlparse-0.5.4-py3-none-any.whl.metadata (4.7 kB)
Requirement already satisfied: six in /opt/conda/lib/python3.12/site-packages
(from ipython-sql) (1.17.0)
Requirement already satisfied: ipython-genutils in
/opt/conda/lib/python3.12/site-packages (from ipython-sql) (0.2.0)
Requirement already satisfied: wcwidth in /opt/conda/lib/python3.12/site-
packages (from prettytable) (0.2.13)
Requirement already satisfied: greenlet!=0.4.17 in
/opt/conda/lib/python3.12/site-packages (from sqlalchemy>=2.0->ipython-sql)
```

```
(3.1.1)
Requirement already satisfied: typing-extensions>=4.6.0 in
/opt/conda/lib/python3.12/site-packages (from sqlalchemy>=2.0->ipython-sql)
(4.12.2)
Requirement already satisfied: decorator in /opt/conda/lib/python3.12/site-
packages (from ipython->ipython-sql) (5.1.1)
Requirement already satisfied: jedi>=0.16 in /opt/conda/lib/python3.12/site-
packages (from ipython->ipython-sql) (0.19.2)
Requirement already satisfied: matplotlib-inline in
/opt/conda/lib/python3.12/site-packages (from ipython->ipython-sql) (0.1.7)
Requirement already satisfied: pexpect>4.3 in /opt/conda/lib/python3.12/site-
packages (from ipython->ipython-sql) (4.9.0)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
/opt/conda/lib/python3.12/site-packages (from ipython->ipython-sql) (3.0.50)
Requirement already satisfied: pygments>=2.4.0 in
/opt/conda/lib/python3.12/site-packages (from ipython->ipython-sql) (2.19.1)
Requirement already satisfied: stack_data in /opt/conda/lib/python3.12/site-
packages (from ipython->ipython-sql) (0.6.3)
Requirement already satisfied: traitlets>=5.13.0 in
/opt/conda/lib/python3.12/site-packages (from ipython->ipython-sql) (5.14.3)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
/opt/conda/lib/python3.12/site-packages (from jedi>=0.16->ipython->ipython-sql)
(0.8.4)
Requirement already satisfied: ptyprocess>=0.5 in
/opt/conda/lib/python3.12/site-packages (from pexpect>4.3->ipython->ipython-sql)
(0.7.0)
Requirement already satisfied: executing>=1.2.0 in
/opt/conda/lib/python3.12/site-packages (from stack_data->ipython->ipython-sql)
(2.1.0)
Requirement already satisfied: asttokens>=2.1.0 in
/opt/conda/lib/python3.12/site-packages (from stack_data->ipython->ipython-sql)
(3.0.0)
Requirement already satisfied: pure_eval in /opt/conda/lib/python3.12/site-
packages (from stack_data->ipython->ipython-sql) (0.2.3)
Downloading ipython_sql-0.5.0-py3-none-any.whl (20 kB)
Downloading prettytable-3.17.0-py3-none-any.whl (34 kB)
Downloading sqlparse-0.5.4-py3-none-any.whl (45 kB)
Installing collected packages: sqlparse, prettytable, ipython-sql
Successfully installed ipython-sql-0.5.0 prettytable-3.17.0 sqlparse-0.5.4
```

1.1.5 Store the datasets in database tables

To analyze the data using SQL, it first needs to be loaded into SQLite DB. We will create three tables in as under:

1. **CENSUS_DATA**
2. **CHICAGO_PUBLIC_SCHOOLS**
3. **CHICAGO_CRIME_DATA**

Load the pandas and sqlite3 libraries and establish a connection to FinalDB.db

[2]:
```
import pandas as pd
import sqlite3

# Create or connect to the SQLite database
conn = sqlite3.connect('FinalDB.db')

# Optional: create a cursor if you need to run SQL commands
cursor = conn.cursor()

print("Connection to FinalDB.db established successfully.")
```

```
Connection to FinalDB.db established successfully.
```

Load the SQL magic module

[3]:
```
%load_ext sql
```

Use Pandas to load the data available in the links above to dataframes. Use these dataframes to load data on to the database FinalDB.db as required tables.

[4]:
```
import pandas as pd
import sqlite3

# ---------------------------------------------------
# 1. Connect to the SQLite database
# ---------------------------------------------------
conn = sqlite3.connect("FinalDB.db")

# ---------------------------------------------------
# 2. Load the CSV files from the provided URLs
# ---------------------------------------------------
census_url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
↪cloud/IBMDeveloperSkillsNetwork-DB0201EN-SkillsNetwork/labs/
↪FinalModule_Coursera_V5/data/ChicagoCensusData.csv"
schools_url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.
↪cloud/IBMDeveloperSkillsNetwork-DB0201EN-SkillsNetwork/labs/
↪FinalModule_Coursera_V5/data/ChicagoPublicSchools.csv"
crime_url = "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
↪IBMDeveloperSkillsNetwork-DB0201EN-SkillsNetwork/labs/
↪FinalModule_Coursera_V5/data/ChicagoCrimeData.csv"

df_census = pd.read_csv(census_url)
df_schools = pd.read_csv(schools_url)
df_crime = pd.read_csv(crime_url)

# ---------------------------------------------------
# 3. Load the DataFrames into SQLite tables
```

```
# ---------------------------------------------------
df_census.to_sql("CENSUS_DATA", conn, if_exists="replace", index=False)
df_schools.to_sql("CHICAGO_PUBLIC_SCHOOLS", conn, if_exists="replace",␣
↪index=False)
df_crime.to_sql("CHICAGO_CRIME_DATA", conn, if_exists="replace", index=False)

# ---------------------------------------------------
# 4. Close the database connection
# ---------------------------------------------------
conn.close()

print("All online datasets successfully loaded into FinalDB.db.")
```

```
All online datasets successfully loaded into FinalDB.db.
```

Establish a connection between SQL magic module and the database `FinalDB.db`

You can now proceed to the the following questions. Please note that a graded assignment will follow this lab and there will be a question on each of the problems stated below. It can be from the answer you received or the code you write for this problem. Therefore, please keep a note of both your codes as well as the response you generate.

1.2 Problems

Now write and execute SQL queries to solve assignment problems

1.2.1 Problem 1

Find the total number of crimes recorded in the CRIME table.

[6]:
```
%sql sqlite:///FinalDB.db

%sql SELECT COUNT(*) AS TOTAL_CRIMES FROM CHICAGO_CRIME_DATA;
```

```
 * sqlite:///FinalDB.db
Done.
```

[6]:
```
[(533,)]
```

1.2.2 Problem 2

List community area names and numbers with per capita income less than 11000.

[9]:
```
%sql SELECT COMMUNITY_AREA_NUMBER, COMMUNITY_AREA_NAME, PER_CAPITA_INCOME FROM␣
↪CENSUS_DATA WHERE PER_CAPITA_INCOME < 11000;
```

```
 * sqlite:///FinalDB.db
Done.
```

[9]:
```
[(26.0, 'West Garfield Park', 10934),
 (30.0, 'South Lawndale', 10402),
 (37.0, 'Fuller Park', 10432),
 (54.0, 'Riverdale', 8201)]
```

1.2.3 Problem 3

List all case numbers for crimes involving minors?(children are not considered minors for the purposes of crime analysis)

[10]:
```
%sql SELECT CASE_NUMBER FROM CHICAGO_CRIME_DATA WHERE DESCRIPTION LIKE␣
↪'%MINOR%' AND DESCRIPTION NOT LIKE '%CHILD%';
```

```
 * sqlite:///FinalDB.db
Done.
```

[10]:
```
[('HL266884',), ('HK238408',)]
```

1.2.4 Problem 4

List all kidnapping crimes involving a child?

[11]:
```
%sql SELECT DISTINCT PRIMARY_TYPE FROM CHICAGO_CRIME_DATA WHERE␣
↪LOCATION_DESCRIPTION LIKE '%SCHOOL%';
```

```
 * sqlite:///FinalDB.db
Done.
```

[11]:
```
[('BATTERY',),
 ('CRIMINAL DAMAGE',),
 ('NARCOTICS',),
 ('ASSAULT',),
 ('CRIMINAL TRESPASS',),
 ('PUBLIC PEACE VIOLATION',)]
```

1.2.5 Problem 5

List the kind of crimes that were recorded at schools. (No repetitions)

[12]:
```
%sql SELECT DISTINCT PRIMARY_TYPE FROM CHICAGO_CRIME_DATA WHERE␣
↪LOCATION_DESCRIPTION LIKE '%SCHOOL%';
```

```
 * sqlite:///FinalDB.db
Done.
```

[12]:
```
[('BATTERY',),
 ('CRIMINAL DAMAGE',),
 ('NARCOTICS',),
 ('ASSAULT',),
 ('CRIMINAL TRESPASS',),
 ('PUBLIC PEACE VIOLATION',)]
```

1.2.6 Problem 6

List the type of schools along with the average safety score for each type.

[16]:
```
#Identify the correct columns
%sql PRAGMA table_info(CHICAGO_PUBLIC_SCHOOLS);
```

```
 * sqlite:///FinalDB.db
Done.
```

[16]:
```
[(0, 'School_ID', 'INTEGER', 0, None, 0),
 (1, 'NAME_OF_SCHOOL', 'TEXT', 0, None, 0),
 (2, 'Elementary, Middle, or High School', 'TEXT', 0, None, 0),
 (3, 'Street_Address', 'TEXT', 0, None, 0),
 (4, 'City', 'TEXT', 0, None, 0),
 (5, 'State', 'TEXT', 0, None, 0),
 (6, 'ZIP_Code', 'INTEGER', 0, None, 0),
 (7, 'Phone_Number', 'TEXT', 0, None, 0),
 (8, 'Link', 'TEXT', 0, None, 0),
 (9, 'Network_Manager', 'TEXT', 0, None, 0),
 (10, 'Collaborative_Name', 'TEXT', 0, None, 0),
 (11, 'Adequate_Yearly_Progress_Made_', 'TEXT', 0, None, 0),
 (12, 'Track_Schedule', 'TEXT', 0, None, 0),
 (13, 'CPS_Performance_Policy_Status', 'TEXT', 0, None, 0),
 (14, 'CPS_Performance_Policy_Level', 'TEXT', 0, None, 0),
 (15, 'HEALTHY_SCHOOL_CERTIFIED', 'TEXT', 0, None, 0),
 (16, 'Safety_Icon', 'TEXT', 0, None, 0),
 (17, 'SAFETY_SCORE', 'REAL', 0, None, 0),
 (18, 'Family_Involvement_Icon', 'TEXT', 0, None, 0),
 (19, 'Family_Involvement_Score', 'TEXT', 0, None, 0),
 (20, 'Environment_Icon', 'TEXT', 0, None, 0),
 (21, 'Environment_Score', 'REAL', 0, None, 0),
 (22, 'Instruction_Icon', 'TEXT', 0, None, 0),
 (23, 'Instruction_Score', 'REAL', 0, None, 0),
 (24, 'Leaders_Icon', 'TEXT', 0, None, 0),
 (25, 'Leaders_Score', 'TEXT', 0, None, 0),
 (26, 'Teachers_Icon', 'TEXT', 0, None, 0),
 (27, 'Teachers_Score', 'TEXT', 0, None, 0),
 (28, 'Parent_Engagement_Icon', 'TEXT', 0, None, 0),
 (29, 'Parent_Engagement_Score', 'TEXT', 0, None, 0),
 (30, 'Parent_Environment_Icon', 'TEXT', 0, None, 0),
 (31, 'Parent_Environment_Score', 'TEXT', 0, None, 0),
 (32, 'AVERAGE_STUDENT_ATTENDANCE', 'TEXT', 0, None, 0),
 (33, 'Rate_of_Misconducts__per_100_students_', 'REAL', 0, None, 0),
 (34, 'Average_Teacher_Attendance', 'TEXT', 0, None, 0),
 (35, 'Individualized_Education_Program_Compliance_Rate', 'TEXT', 0, None, 0),
 (36, 'Pk_2_Literacy__', 'TEXT', 0, None, 0),
 (37, 'Pk_2_Math__', 'TEXT', 0, None, 0),
 (38, 'Gr3_5_Grade_Level_Math__', 'TEXT', 0, None, 0),
 (39, 'Gr3_5_Grade_Level_Read__', 'TEXT', 0, None, 0),
 (40, 'Gr3_5_Keep_Pace_Read__', 'TEXT', 0, None, 0),
 (41, 'Gr3_5_Keep_Pace_Math__', 'TEXT', 0, None, 0),
 (42, 'Gr6_8_Grade_Level_Math__', 'TEXT', 0, None, 0),
 (43, 'Gr6_8_Grade_Level_Read__', 'TEXT', 0, None, 0),
```

```
 (44, 'Gr6_8_Keep_Pace_Math_', 'TEXT', 0, None, 0),
 (45, 'Gr6_8_Keep_Pace_Read__', 'TEXT', 0, None, 0),
 (46, 'Gr_8_Explore_Math__', 'TEXT', 0, None, 0),
 (47, 'Gr_8_Explore_Read__', 'TEXT', 0, None, 0),
 (48, 'ISAT_Exceeding_Math__', 'REAL', 0, None, 0),
 (49, 'ISAT_Exceeding_Reading__', 'REAL', 0, None, 0),
 (50, 'ISAT_Value_Add_Math', 'REAL', 0, None, 0),
 (51, 'ISAT_Value_Add_Read', 'REAL', 0, None, 0),
 (52, 'ISAT_Value_Add_Color_Math', 'TEXT', 0, None, 0),
 (53, 'ISAT_Value_Add_Color_Read', 'TEXT', 0, None, 0),
 (54, 'Students_Taking__Algebra__', 'TEXT', 0, None, 0),
 (55, 'Students_Passing__Algebra__', 'TEXT', 0, None, 0),
 (56, '9th Grade EXPLORE (2009)', 'TEXT', 0, None, 0),
 (57, '9th Grade EXPLORE (2010)', 'TEXT', 0, None, 0),
 (58, '10th Grade PLAN (2009)', 'TEXT', 0, None, 0),
 (59, '10th Grade PLAN (2010)', 'TEXT', 0, None, 0),
 (60, 'Net_Change_EXPLORE_and_PLAN', 'TEXT', 0, None, 0),
 (61, '11th Grade Average ACT (2011)', 'TEXT', 0, None, 0),
 (62, 'Net_Change_PLAN_and_ACT', 'TEXT', 0, None, 0),
 (63, 'College_Eligibility__', 'TEXT', 0, None, 0),
 (64, 'Graduation_Rate__', 'TEXT', 0, None, 0),
 (65, 'College_Enrollment_Rate__', 'TEXT', 0, None, 0),
 (66, 'COLLEGE_ENROLLMENT', 'INTEGER', 0, None, 0),
 (67, 'General_Services_Route', 'INTEGER', 0, None, 0),
 (68, 'Freshman_on_Track_Rate__', 'TEXT', 0, None, 0),
 (69, 'X_COORDINATE', 'REAL', 0, None, 0),
 (70, 'Y_COORDINATE', 'REAL', 0, None, 0),
 (71, 'Latitude', 'REAL', 0, None, 0),
 (72, 'Longitude', 'REAL', 0, None, 0),
 (73, 'COMMUNITY_AREA_NUMBER', 'INTEGER', 0, None, 0),
 (74, 'COMMUNITY_AREA_NAME', 'TEXT', 0, None, 0),
 (75, 'Ward', 'INTEGER', 0, None, 0),
 (76, 'Police_District', 'INTEGER', 0, None, 0),
 (77, 'Location', 'TEXT', 0, None, 0)]
```

[17]:
```
%sql SELECT "Elementary, Middle, or High School" AS School_Type,␣
↪AVG(SAFETY_SCORE) AS Average_Safety_Score FROM CHICAGO_PUBLIC_SCHOOLS GROUP␣
↪BY "Elementary, Middle, or High School";
```

```
 * sqlite:///FinalDB.db
Done.
```

[17]:
```
[('ES', 49.52038369304557), ('HS', 49.62352941176471), ('MS', 48.0)]
```

1.2.7 Problem 7

List 5 community areas with highest % of households below poverty line

[18]:
```
%sql SELECT COMMUNITY_AREA_NUMBER, COMMUNITY_AREA_NAME,
  ↪PERCENT_HOUSEHOLDS_BELOW_POVERTY FROM CENSUS_DATA ORDER BY
  ↪PERCENT_HOUSEHOLDS_BELOW_POVERTY DESC LIMIT 5;
```

```
 * sqlite:///FinalDB.db
Done.
```

[18]:
```
[(54.0, 'Riverdale', 56.5),
 (37.0, 'Fuller Park', 51.2),
 (68.0, 'Englewood', 46.6),
 (29.0, 'North Lawndale', 43.1),
 (27.0, 'East Garfield Park', 42.4)]
```

1.2.8 Problem 8

Which community area is most crime prone? Display the coumminty area number only.

[19]:
```
%sql SELECT COMMUNITY_AREA_NUMBER FROM CHICAGO_CRIME_DATA GROUP BY
  ↪COMMUNITY_AREA_NUMBER ORDER BY COUNT(*) DESC LIMIT 1;
```

```
 * sqlite:///FinalDB.db
Done.
```

[19]:
```
[(25.0,)]
```

Double-click **here** for a hint

1.2.9 Problem 9

Use a sub-query to find the name of the community area with highest hardship index

[20]:
```
%sql SELECT COMMUNITY_AREA_NAME FROM CENSUS_DATA WHERE HARDSHIP_INDEX = (SELECT
  ↪MAX(HARDSHIP_INDEX) FROM CENSUS_DATA);
```

```
 * sqlite:///FinalDB.db
Done.
```

[20]:
```
[('Riverdale',)]
```

1.2.10 Problem 10

Use a sub-query to determine the Community Area Name with most number of crimes?

[21]:
```
%sql SELECT COMMUNITY_AREA_NAME FROM CENSUS_DATA WHERE COMMUNITY_AREA_NUMBER =
  ↪(SELECT COMMUNITY_AREA_NUMBER FROM CHICAGO_CRIME_DATA GROUP BY
  ↪COMMUNITY_AREA_NUMBER ORDER BY COUNT(*) DESC LIMIT 1);
```

```
 * sqlite:///FinalDB.db
Done.
```

[21]:
```
[('Austin',)]
```

1.3 Author(s)

Hima Vasudevan

Rav Ahuja

Ramesh Sannreddy

1.4 Contribtuor(s)

Malika Singla

Abhishek Gagneja

##

© IBM Corporation 2023. All rights reserved.